TD : Terminaison d'Algorithmes

Correction Detaillee avec Demonstrations

Olivier Raynaud (raynaud@isima.fr)

Version corrigee et enrichie

Version 2025

Contents

1 Introduction : Concepts de Terminaison **2**

2 Exercice 1 : Tests d'Arret des Algorithmes **3**
2.1 Algorithme 1 : Factorielle 3
2.2 Algorithme 2 : Diverge 4
2.3 Algorithme 3 : Converge 5

3 Exercice 2 : Le Drapeau Hollandais **6**
3.1 Question 1 : Complexite et Terminaison 6

4 Exercice 3 : PGCD d'Euclide **8**

5 Exercice 4 : Entiers Signes dans un Tableau **10**

6 Exercice 5 : Probleme des Pancakes **12**

7 Exercice 6 : Mariages Stables (Gale-Shapley) **14**
7.1 Question 1 : Preuve de Terminaison 14
7.2 Question 2 : Preuve de Stabilite 15

8 Synthese et Conseils **16**

1 Introduction : Concepts de Terminaison

Note de Cours

Qu'est-ce que la terminaison d'un algorithme ?
Un algorithme **termine** (ou **s'arrete**) si pour toute entree valide, il atteint eventuellement son instruction de retour apres un nombre fini d'etapes.
Pourquoi est-ce important ?

- Un algorithme qui ne termine pas est **inutile** en pratique
- La terminaison est une propriete fondamentale de correction
- Elle garantit qu'on obtiendra effectivement un resultat

Important

Methodes principales pour prouver la terminaison :

1. **Variant de boucle :**
   - Trouver une fonction V a valeurs dans $\mathbb{N}$ (ou ensemble bien fonde)
   - Montrer que V **decroit strictement** a chaque iteration
   - Comme $\mathbb{N}$ est bien fonde, la boucle termine
2. **Mesure de progres :**
   - Identifier une quantite qui se rapproche de la condition d'arret
   - Montrer qu'elle evolue monotonement
   - Prouver qu'elle atteint la condition d'arret
3. **Borne sur le nombre d'iterations :**
   - Majorer directement le nombre d'iterations possibles
   - Montrer qu'il est fini

2 Exercice 1 : Tests d'Arret des Algorithmes

2.1 Algorithme 1 : Factorielle

Exemple

```
Donnees: N : entier
Resultat: f : entier (factorielle de N)
debut
    n <- N ; f <- fact(7)
    tant que (n != 7) faire
        si (n > 7) alors
            f <- f * n ; n <- n - 1
        sinon
            n <- n + 1 ; f <- f / n
        fin
    fin
    retourner f
fin
```

Solution

Analyse de la terminaison :
Variant de boucle : $V(n) = |n - 7|$ (distance a 7)
Cas 1 : Si $n > 7$: $n \leftarrow n - 1$, donc $V(n_{nouveau}) = n - 8 < n - 7 = V(n_{ancien})$
Cas 2 : Si $n < 7$: $n \leftarrow n + 1$, donc $V(n_{nouveau}) = 6 - n < 7 - n = V(n_{ancien})$
Cas 3 : Si $n = 7$: la boucle s'arrete.
Conclusion : $V(n)$ decroit strictement, $V(n) \geq 0$. Par suite decroissante dans $\mathbb{N}$, **l'algorithme termine toujours.**

Note de Cours

Remarque : Bien que l'algorithme termine, pour $n < 7$ on divise par n croissant ce qui ne calcule pas correctement la factorielle. *Terminaison $\neq$ Correction !*

Correction du Professeur (Raynaud)

Le prof etudie la **suite** $\mathcal{N}$ des valeurs de la variable n. Il distingue trois cas :

- Si $N = 7$: $n_0 = 7$, la suite s'arrete sur n_0 immediatement.
- Si $N > 7$: la suite est **minoree par 7** et arithmetique de raison -1, elle converge donc vers 7.
- Si $N < 7$: la suite est **majoree par 7** et arithmetique de raison $+1$, elle converge donc vers 7.

Conclusion du prof : Dans les trois cas la suite converge vers 7 (valeur qui permet de stopper l'algorithme). Le test d'arret est verifie.

Difference : Le prof raisonne par "suite arithmetique convergente" plutot que par variant $|n-7|$. Les deux approches sont equivalentes, mais celle du prof est plus concise.

2.2 Algorithme 2 : Diverge

Exemple

```
Donnees: N : entier
Resultat: n : entier
debut
    n <- N
    tant que (n != 0) faire
        si (n est pair) alors
            n <- 2 * n
        sinon
            n <- n - 1
        fin
    fin
    retourner n
fin
```

Solution

Etude par cas :
Cas 1 : $N = 0$: condition fausse immediatement. **Termine.**
Cas 2 : $N < 0$: n s'eloigne de 0 indefiniment. **Ne termine jamais.**
Cas 3 : $N > 0$ impair : $n \to n-1$ (pair), puis doublement infini. **Ne termine jamais** (sauf $N = 1$).
Cas 4 : $N > 0$ pair : doublement infini. **Ne termine jamais.**
Conclusion : L'algorithme termine uniquement pour $N \in \{0, 1\}$.

Correction du Professeur (Raynaud)

Le prof etudie aussi par cas mais avec une analyse plus rapide :

- $N = 0$: $n_0 = 0$, suite finie d'un seul terme.
- $N = 1$: $n_0 = 1$, $n_1 = 0$, suite finie de deux termes.
- N pair : la suite **diverge vers** $+\infty$.
- N impair (et $N > 1$) : $n_1 = N - 1$ est pair, donc la suite diverge aussi vers $+\infty$.

Conclusion du prof : "L'algorithme 2 n'est donc pas correct puisque nous ne pouvons pas assurer son arret."
Difference : Le prof conclut directement que l'algorithme n'est **pas correct** (au sens ou il ne termine pas pour toute entree). Notre version ajoute le cas $N = 1$ qui termine, detail que le prof a aussi note.

2.3 Algorithme 3 : Converge

Exemple

```
Donnees: N : entier
Resultat: n : entier
debut
    n <- N
    tant que (n != 1) faire
        si (n est pair) alors
            n <- n / 2
        sinon
            n <- n + 1
        fin
    fin
    retourner n
fin
```

Solution

Cas 1 : $N = 1$: termine immediatement.
Cas 2 : $N \leq 0$: arrive a $0 \to 0 \to \dots$ (boucle infinie). Ne termine pas.
Cas 3 : $N > 1$: Variant $V(n) = n - 1$.

- n pair : $V(n/2) = n/2 - 1 < n - 1 = V(n)$
- n impair : apres deux iterations $n \to n + 1 \to (n + 1)/2$, et $V((n + 1)/2) = (n - 1)/2 < n - 1$

Conclusion : L'algorithme termine si et seulement si $N \geq 1$.

Correction du Professeur (Raynaud)

Le prof utilise une approche plus sophistiquee : il introduit une **suite auxiliaire** $\mathcal{N}'$ definie par :

$$n'_0 = n_0 = N, \quad n'_{i+1} = \lceil n_i/2 \rceil$$

Il montre que tous les termes de $\mathcal{N}'$ appartiennent a $\mathcal{N}$ (la suite des valeurs de n). Puis il argue que $\mathcal{N}'$ est :

- **positive**
- **strictement decroissante**
- de nature **geometrique** de raison strictement inferieure a 1

Elle converge donc vers 1. Comme $\mathcal{N}'$ est une sous-suite de $\mathcal{N}$, la suite $\mathcal{N}$ converge aussi vers 1.
Difference : Le prof ne considere pas le cas $N \leq 0$ (il suppose implicitement $N \geq 1$). Sa methode de **sous-suite auxiliaire geometrique** est plus elegante que notre variant $V = n - 1$.

3 Exercice 2 : Le Drapeau Hollandais

Definition

Probleme du drapeau hollandais :
Entree : Un tableau `tab[1..n]` contenant des couleurs : bleu, blanc, rouge
Sortie : Le tableau `tab[]` range : bleus a gauche, blancs au centre, rouges a droite.

Exemple

```
Donnees: tab[1..n] : tableau de couleurs
Resultat: tab[1..n] : tableau range
Variables: indiceBleu, indiceRouge, compteur : entier
debut
    indiceBleu <- 1 ; indiceRouge <- n
    compteur <- 1
    tant que (compteur <= indiceRouge) faire
        si (tab[compteur] = bleu) alors
            echanger(tab, compteur, indiceBleu)
            indiceBleu <- indiceBleu + 1
            compteur <- compteur + 1
        sinon si (tab[compteur] = rouge) alors
            echanger(tab, compteur, indiceRouge)
            indiceRouge <- indiceRouge - 1
        sinon  // blanc
            compteur <- compteur + 1
        fin
    fin
    retourner tab
fin
```

3.1 Question 1 : Complexite et Terminaison

Solution

A) Complexite : Dans tous les cas (bleu/rouge/blanc), l'ecart `indiceRouge - compteur` diminue de 1. Nombre max d'iterations : n. **Complexite :** $O(n)$, spatiale $O(1)$.
B) Variant de boucle : $V = \text{indiceRouge} - \text{compteur} + 1$

- Bleu : compteur monte de 1 $\Rightarrow$ V diminue de 1
- Rouge : indiceRouge baisse de 1 $\Rightarrow$ V diminue de 1
- Blanc : compteur monte de 1 $\Rightarrow$ V diminue de 1

V decroit strictement, $V \geq 0$. **L'algorithme termine toujours.**

A savoir

Invariants de boucle : A tout moment :

- `tab[1..indiceBleu-1]` : uniquement des bleus
- `tab[indiceRouge+1..n]` : uniquement des rouges
- `tab[indiceBleu..compteur-1]` : uniquement des blancs
- `tab[compteur..indiceRouge]` : non encore examine

Correction du Professeur (Raynaud)

Le prof donne une description plus concise de l'algorithme. Il explique les deux indices :

- **indiceBleu** : toutes les cases d'indice inferieur ou egal sont bleues
- **indiceRouge** : toutes les cases d'indice superieur ou egal sont rouges

Pour la terminaison, le prof indique :
"La complexite de cet algorithme est lineaire dans la taille du tableau. Pour la preuve de terminaison, il suffit d'etudier la suite de valeurs de l'expression **indiceRouge − compteur**. On pourra montrer que cette suite **decroit strictement**."
Difference : Le prof utilise exactement le meme variant que nous (indiceRouge − compteur) mais laisse la verification "a faire". Notre version la fait en detail pour les 3 cas. Le prof note aussi que la condition correcte est `compteur` $<$ `indiceRouge` (dans son enonce original).

4 Exercice 3 : PGCD d'Euclide

Exemple

```
Donnees: N, M : entiers positifs
Resultat: PGCD(N, M)
Variables: n, m : entiers
debut
    n <- N ; m <- M
    tant que (n != 0) faire
        si (m > n) alors
            m <- m - n
        sinon
            n <- n - m
        fin
    fin
    retourner m
fin
```

Note de Cours

Principe : $\mathrm{PGCD}(a, b) = \mathrm{PGCD}(a - b, b)$ si $a > b$. On soustrait repetitivement le plus petit du plus grand.

Preuve

Variant de boucle : $V(n, m) = n + m$
Cas 1 : $m > n$: $m \leftarrow m - n$, donc $V_{nouveau} = n + (m - n) = m < n + m = V_{ancien}$. V diminue de n.
Cas 2 : $m \leq n$: $n \leftarrow n - m$, donc $V_{nouveau} = (n - m) + m = n < n + m = V_{ancien}$. V diminue de m.
Dans les deux cas V decroit strictement. $V \geq 0$ et $V = 0 \Leftrightarrow n = 0$ (condition d'arret).
Conclusion : **L'algorithme termine toujours.**

A savoir

Complexite : Version par soustraction : $O(\max(N, M))$. Version optimisee par modulo : $O(\log \min(N, M))$.

Correction du Professeur (Raynaud)

Le prof choisit un variant **differente** : plutot que $n + m$, il etudie la suite $\mathcal{P}$ des valeurs du **produit** $m \times n$ avant chaque entree dans la boucle.
Il montre que chaque terme s'ecrit $p_t = m_t \cdot n_t$ et que :

- Si $m_t > n_t$: $p_{t+1} = (m_t - n_t) \cdot n_t = p_t - n_t^2$ avec $n_t \neq 0$
- Si $n_t \leq m_t$: $p_{t+1} = (n_t - m_t) \cdot m_t = p_t - m_t^2$ avec $m_t \neq 0$

Dans les deux cas $\mathcal{P}$ est **minoree par 0** et **strictement decroissante**, donc elle converge vers 0. Il existe donc un rang k tel que $n_k = 0$.
Difference : Le prof utilise le **produit** $m \cdot n$ comme variant (diminue de n^2 ou m^2), nous utilisons la **somme** $n + m$ (diminue de n ou m). Les deux fonctionnent. Le variant somme est plus classique et plus simple a expliquer.

5 Exercice 4 : Entiers Signes dans un Tableau

Definition

Probleme : Matrice $n \times m$ d'entiers signes. On peut inverser tous les signes d'une ligne ou colonne. Objectif : toutes les lignes et colonnes ont une somme ≥ 0.

Exemple

```
Donnees: T[1..n][1..m] : tableau d'entiers
Resultat: Toutes les lignes et colonnes ont des sommes >= 0
debut
    tant que (Il existe une ligne ou colonne de somme negative) faire
        Inverser les signes de cette ligne ou colonne
    fin
fin
```

Preuve

Tentative 1 – Variant : nombre de coefficients negatifs
$V = |\{(i,j) : T[i][j] < 0\}|$
Si la somme d'une ligne est negative, le nombre d'elements negatifs $n^- > n^+$ (nombre de positifs). Apres inversion, $\Delta V = n^+ - n^- < 0$. V decroit strictement.

Mais : cette preuve est **incorrecte** en general ! Contre-exemple : $\begin{pmatrix} -10 & 1 \\ 1 & 1 \end{pmatrix}$.

Apres inversion ligne 1, on obtient $\begin{pmatrix} 10 & -1 \\ 1 & 1 \end{pmatrix}$: nombre de negatifs inchange (1 avant, 1 apres).

Tentative 2 – Variant correct : Somme des deficits

$$V = \sum_i \max(0, -S_i^{\text{ligne}}) + \sum_j \max(0, -S_j^{\text{colonne}})$$

$V = 0$ ssi toutes les sommes sont ≥ 0. L'analyse est complexe mais fonctionne.

Correction du Professeur (Raynaud)

Le prof donne la **bonne cle** directement :
"L'objectif est de trouver un parametre P, sorte de compteur numerique qui fixe un etat du systeme, qui permet de mesurer la progression vers la cible."
Il explique pourquoi le nombre de lignes/colonnes negatives **ne marche pas** : "l'exemple de la figure 2 montre que le changement de signes le long d'une ligne peut degrader le parametre."
Le prof choisit comme variant :

$$\boxed{P = \text{la somme de tous les nombres de la matrice}}$$

Argument : En inversant une ligne ou colonne de somme $-s$ (avec $s > 0$), on **augmente** P de $2s$. Donc P ne fait que **croître**.

L'ensemble des situations possibles est de $2^{n \cdot m}$ (chaque ligne/colonne peut etre inversee ou non), donc **fini**. Il existera donc une situation a partir de laquelle P sera suffisamment eleve pour que toutes les sommes soient non negatives.
Difference : Le prof utilise la **somme totale de la matrice** comme variant croissant (bornee par un fini d'etats), ce qui est beaucoup plus elegant que notre analyse complexe. C'est l'approche a retenir pour l'examen.

6 Exercice 5 : Probleme des Pancakes

Definition

Donnees : Tableau `t[1..n]` = permutation de $\{1, \ldots, n\}$. Fonction `inverser(t, i, j)` : inverse les elements entre indices i et j.

Exemple

```
Donnees: t[1..n] : tableau d'entiers (permutation de 1..n)
debut
    tant que (t[1] != 1) faire
        t <- inverser(t, 1, t[1])
    fin
    retourner t
fin
```

Preuve

Si $t[1] = k > 1$, on inverse les k premiers elements, puis $t[1]$ prend la valeur de l'ancien $t[k]$.
Exemple : $(4, 7, 3, 2, 5, 6, 1) \to (2, 3, 7, 4, 5, 6, 1) \to (3, 2, 7, 4, 5, 6, 1) \to (7, 2, 3, 4, 5, 6, 1) \to (1, 6, 5, 4, 3, 2, 7)$: arret.
Argument de terminaison : Il y a un nombre fini de permutations ($n!$). L'operation est deterministe et modifie le tableau a chaque iteration. On ne peut pas cycler indefiniment, donc on atteint $t[1] = 1$.

Correction du Professeur (Raynaud)

Le prof introduit un tableau **indicateur** $s[]$ de taille n :

$$s[i] = \begin{cases} 1 & \text{si } t[i] = i \\ 0 & \text{sinon} \end{cases}$$

Pour l'execution de son exemple :

- $(6, 1, 2, 5, 3, 4) \to s = 000000$
- $(4, 3, 5, 2, 1, 6) \to s = 000001$
- $(2, 5, 3, 4, 1, 6) \to s = 011101$ (note : typo dans le PDF, c'est s pour cette permutation)
- $(1, 4, 3, 2, 5, 6) \to s = 100011$

Observation cle du prof : Le tableau s **augmente strictement** pour l'ordre **lexicographique** a chaque transformation.
Conclusion du prof : L'algorithme execute donc **moins de** 2^n passages dans la boucle (car s est un vecteur binaire de taille n, donc 2^n valeurs possibles).

Difference : Le prof donne une **borne explicite** (2^n iterations) en utilisant l'ordre lexicographique sur $s[]$, alors que notre preuve donne seulement la borne $n!$. La borne 2^n est **beaucoup meilleure**. Le prof mentionne aussi que la meilleure borne reste un **probleme ouvert** (probleme des pancakes).

7 Exercice 6 : Mariages Stables (Gale-Shapley)

Definition

Probleme : n hommes et n femmes avec preferences strictes. Trouver un **mariage stable** (pas de couple instable).

Exemple

```
Donnees: Listes de preferences de chaque homme et femme
Resultat: Mariage stable
debut
    tant que (Il existe un homme libre n'ayant pas courtise chaque femme) faire
        Choisir m un tel homme
        Soit w la premiere femme non courtisee de m
        si (w est libre) alors
            Apparier m et w
        sinon
            si (w prefere m a son partenaire courant m') alors
                Apparier m et w ; Liberer m'
            sinon
                w rejette m
            fin
        fin
    fin
    retourner le mariage
fin
```

7.1 Question 1 : Preuve de Terminaison

Preuve

Variant : $V = \sum_h$(nombre de femmes non encore courtisees par h)
$V_0 = n^2$. A chaque iteration, un homme courtise une **nouvelle** femme, donc V diminue de 1. $V \geq 0$, donc la boucle termine apres au plus n^2 iterations.
Lemme : Si un homme est libre, il existe une femme libre (car les deux populations ont la meme taille). Donc le cas 2 (homme libre ayant tout courtise) ne se produit pas.
L'algorithme termine toujours en au plus n^2 iterations.

Correction du Professeur (Raynaud)

Le prof ajoute deux **remarques importantes** avant la preuve :

- **Remarque 1 :** Les hommes font leur demandes par ordre de preference **decroissante**.
- **Remarque 2 :** Une fois qu'une femme a un partenaire, elle n'est jamais **liberee**, elle ne peut etre que **reaffectee** a un homme qu'elle prefere.

Le prof enonce et prouve une **proposition formelle** :
"L'algorithme s'arrete avant n^2 iterations de la boucle tant que."
Preuve du prof : A chaque passage dans la boucle, un homme fait sa demande a une **nouvelle** femme. Il y a seulement n^2 demandes possibles (precisement $n \times (n-1)+1$). L'algorithme s'arrete lorsque chaque femme est appariee. Si une femme a au moins une demande elle est appariee. La borne est atteinte pour $n = 2$.
Difference : Le prof donne la borne plus precise $n(n-1)+1$ au lieu de n^2, et explique que cette borne est **atteinte pour** $n = 2$ mais pas au-dela.

7.2 Question 2 : Preuve de Stabilite

Preuve

Supposons par contradiction un couple instable (h, f) : h prefere f a f' et f prefere h a h'.
Cas 1 : h a courtise f. Alors f a rejete h (ou l'a remplace). Comme les partenaires des femmes s'ameliorent au cours de l'algorithme, f prefere h' a h. Contradiction.
Cas 2 : h n'a jamais courtise f. Alors h a ete apparie avec une femme qu'il prefere a f, puis libere. Dans ce cas il aurait courtise f avant f'. Contradiction.
Le mariage est stable.

Correction du Professeur (Raynaud)

Le prof donne exactement la meme structure de preuve par contradiction, avec les deux cas :

- "H n'a jamais fait sa demande a F : autrement dit H prefere sa partenaire actuelle donc (H, F) est stable."
- "H a fait sa demande a F : F a rejete H (lors de sa demande ou plus tard), F prefere donc son partenaire actuel a H, (H, F) est donc stable."

La conclusion est la meme : contradiction dans les deux cas, donc pas de couple instable.
Le prof ajoute aussi deux **questions pour aller plus loin** :

- Comment implementer **efficacement** l'algorithme ?
- Si plusieurs appariements stables existent, **lequel** produit l'algorithme ? (Reponse : celui **optimal pour les hommes**, pessimal pour les femmes.)

8 Synthese et Conseils

Important

Methodes pour prouver la terminaison :

1. **Variant de boucle** : fonction V : Etats $\to \mathbb{N}$, strictement decroissante.
2. **Borne sur le nombre d'iterations** : majorer et montrer que c'est fini.
3. **Ensemble bien fonde** : ordre lexicographique, etc.

A savoir

Astuces pour trouver un variant :

1. **Distance a l'objectif**
2. **Ressources consommees** (ex : courtisages restants)
3. **Taille des structures**
4. **Valeur numerique** qui decroit
5. **Ordre lexicographique** sur des vecteurs (ex : pancakes)
6. **Somme totale** comme potentiel croissant (ex : matrice signes)

Note de Cours

Erreurs courantes :

- Confondre terminaison et correction
- Oublier que le variant doit decroître **strictement**
- Choisir un variant qui peut devenir negatif
- Ne pas considerer tous les cas d'execution

Conclusion – Resume

Resume des resultats

Algorithme	**Termine?**	**Variant (nous)**	**Variant (prof)**
Factorielle	Oui	$\lvert n-7\rvert$	Suite arithmetique vers 7
Diverge	Non ($\neq 0, 1$)	N/A	N/A
Converge	Oui ($N \geq 1$)	$n-1$	Sous-suite geometrique
Drapeau Hollandais	Oui	indiceRouge$-$compteur	Meme (a faire)
PGCD Euclide	Oui	$n+m$	Produit $m \cdot n$
Inversion matrice	Oui	Somme des deficits	Somme totale (croissante)
Pancakes	Oui	$n!$ configs	Ordre lex sur $s[]$ (2^n)
Gale-Shapley	Oui	Courtisages restants	Meme, borne $n(n-1)+1$

Bon courage pour vos examens !